



Universidad Autónoma de Querétaro
Facultad de Contaduría y Administración



Proyecto final

Por:

Estudillo de Anda Luis Angel

SQL

Introducción

SQL (Structured Query Language) es un lenguaje de programación estándar utilizado para interactuar con bases de datos relacionales. Fue desarrollado en los años 70 por IBM y ha sido adoptado ampliamente debido a su capacidad para gestionar datos de manera eficiente.

SQL permite realizar operaciones como:

- Crear y modificar estructuras de datos: Bases de datos, tablas e índices.
- Manipular datos: Insertar, actualizar, consultar y eliminar información.
- Administrar bases de datos: Definir permisos de acceso y gestionar usuarios.

Conceptos Clave

- Base de Datos Relacional: Colección de tablas que organizan datos en filas (registros) y columnas (atributos).
- Tabla: Estructura bidimensional formada por columnas (campos) y filas (registros).
- Registro: Fila que representa una instancia de una entidad.
- Clave Primaria: Identificador único para un registro dentro de una tabla.
- Clave Foránea: Campo en una tabla que establece una relación con la clave primaria de otra tabla.

Comandos Básicos de SQL

Crear una tabla

El comando CREATE TABLE se utiliza para definir la estructura de una nueva tabla, incluyendo sus columnas y tipos de datos.

Ejemplo:

```
CREATE TABLE Empleados (  
    ID INT PRIMARY KEY,  
    Nombre VARCHAR(50),  
    Puesto VARCHAR(30)  
);
```

Insertar datos

El comando INSERT INTO permite agregar nuevos registros a una tabla.

Ejemplo:

```
INSERT INTO Empleados (ID, Nombre, Puesto)  
VALUES (1, 'Ana Pérez', 'Gerente');
```

Consultar datos

El comando SELECT permite recuperar datos de una tabla.

Ejemplo:

```
SELECT Nombre, Puesto  
FROM Empleados;
```

Actualizar datos

El comando UPDATE modifica registros existentes en una tabla.

Ejemplo:

```
UPDATE Empleados  
SET Puesto = 'Director'  
WHERE ID = 1;
```

Eliminar datos

El comando DELETE elimina registros específicos de una tabla.

Ejemplo:

```
DELETE FROM Empleados  
WHERE ID = 1;
```

Normalización

La normalización es un proceso para organizar datos en una base de datos, minimizando la redundancia y asegurando la integridad de los datos.

Formas normales:

- Primera Forma Normal (1NF): Elimina grupos repetidos asegurando que cada celda contenga un único valor.
- Segunda Forma Normal (2NF): Elimina dependencias parciales asegurando que cada atributo dependa completamente de la clave primaria.
- Tercera Forma Normal (3NF): Elimina dependencias transitivas asegurando que los atributos dependan solo de la clave primaria.

Base de Datos vs Conjunto de Datos

Base de Datos	Conjunto de Datos
----------------------	--------------------------

Propósito: Almacenamiento, recuperación y manipulación.	Propósito: Análisis e informes.
---	---------------------------------

Estructura: Tablas, índices, vistas y procedimientos.	Estructura: Tabular.
---	----------------------

Formatos de almacenamiento: MySQL, PostgreSQL, Oracle.	Formatos de almacenamiento: CSV, XLSX, JSON.
--	--

Comando SELECT

El comando principal en DQL es el SELECT, que permite a los usuarios recuperar datos de las tablas de la base de datos. Este comando puede ser utilizado para realizar consultas simples o complejas, dependiendo de las necesidades del usuario. La sintaxis básica de un comando SELECT es:

SELECT columna1, columna2, ... = Especifica las columnas que se desean consultar

FROM nombre_tabla = Indica la tabla de la cual se extraerán los datos.

WHERE condiciones; . = Aplique filtros para restringir los resultados según las condiciones definidas por el usuario.

Tipos de Modelos de Base de Datos

1. Modelo Jerárquico (1950): Los datos están organizados en una estructura de árbol. Las tablas tienen una relación de tipo padre-hijo. Cada tabla hija tiene un único padre, mientras que cada padre puede tener múltiples hijos.

2. Modelo de Red (1960): Los datos se organizan en una estructura de grafo, con nodos y enlaces. Permite que las tablas hijas tengan múltiples padres.

3. Modelo Relacional (1970): Los datos se organizan en tablas con filas (registros) y columnas (campos). Supera las limitaciones de la estructura jerárquica. Las tablas pueden accederse de manera directa sin necesidad de recorrer todos los objetos padres.

4. Modelo Orientado a Objetos (1990): Los datos se representan como objetos, similar a la programación orientada a objetos. Utiliza una estructura tridimensional. Eficiente para la localización de elementos individuales, pero con un rendimiento menor cuando se necesitan varios elementos a la vez.

OTROS CONCEPTOS VISTOS

Registro: Fila que contiene los datos de una observación o entidad.

- Campo: Columna que almacena un dato específico para todos los registros en la tabla.
- Tabla: Estructura bidimensional formada por campos (columnas) y registros (filas). Los campos se definen al crear la tabla y no existe un límite para la cantidad de registros.
- Base de Datos Relacional: Conjunto de tablas relacionadas que almacenan datos de manera estructurada, con relaciones definidas entre ellas.
- Identificadores Únicos: Valores únicos, generalmente enteros, usados para identificar registros dentro de una tabla (también conocidas como Claves Primarias).
- Clave Foránea: Campo (o conjunto de campos) en una tabla que identifica de manera única una fila en otra tabla, estableciendo una relación entre ambas.
- Consulta: Petición de datos o información de una base de datos, escrita en SQL. Las consultas pueden recuperar, insertar, actualizar o eliminar datos.

Diseño de bases de datos

El diseño de bases de datos es un aspecto esencial en la creación de sistemas que sean eficientes y fáciles de escalar. Un diseño bien estructurado garantiza que los datos se almacenen de forma ordenada y coherente, lo que facilita su consulta, actualización y manejo en el futuro. Para lograrlo, es fundamental tener en cuenta la integridad de los datos, la capacidad de escalabilidad y el rendimiento del sistema. Este tema abarca todas las etapas necesarias, desde la recolección de información hasta procesos como la normalización y la desnormalización.

Recolección de requisitos

El primer paso en el diseño de una base de datos es recopilar los requisitos necesarios. Esto es crucial, ya que define la estructura básica del sistema. Durante esta etapa, es importante identificar qué datos se gestionarán, cómo se usarán y qué reglas deben aplicarse. Algunas preguntas clave a considerar son:

1. ¿Qué datos se almacenarán?

Se debe determinar qué entidades serán gestionadas (como clientes, productos o pedidos) y cuáles serán sus atributos (por ejemplo, nombre, precio o fecha de pedido). Esta información permite estructurar las tablas y definir las relaciones entre ellas.

2. ¿Cómo se relacionan las entidades?

Es esencial identificar cómo interactúan las distintas entidades. Por ejemplo, un cliente puede realizar varios pedidos, pero cada pedido solo pertenece a un cliente. También es necesario definir el tipo de relación: uno a uno, uno a muchos o muchos a muchos, para estructurar adecuadamente la base de datos.

3. ¿Qué restricciones deben aplicarse?

Se deben establecer reglas que los datos deben cumplir, como la unicidad (por ejemplo, que el correo de un cliente sea único) o la obligatoriedad de ciertos campos (como el nombre). Estas restricciones aseguran la consistencia y validez de los datos.

Modelado conceptual

Una vez definidos los requisitos, se pasa al modelado conceptual. Esta fase implica la creación de un modelo abstracto que represente la estructura de los datos y sus relaciones. Una herramienta comúnmente utilizada es el Diagrama Entidad-Relación (ER), que muestra visualmente las entidades, atributos y relaciones.

Entidades

Las entidades representan objetos o conceptos relevantes para el sistema que pueden identificarse de manera única. Algunos ejemplos son:

- Clientes: Incluye atributos como ID_Cliente, Nombre y Correo Electrónico.
- Productos: Puede tener atributos como ID_Producto, Nombre y Precio.
- Pedidos: Atributos como ID_Pedido, Fecha y Estado.

Relaciones

Las relaciones muestran cómo interactúan las entidades. Por ejemplo:

- Un cliente puede realizar varios pedidos, pero cada pedido pertenece a un solo cliente (relación uno a muchos).

- Un pedido puede contener varios productos, y un producto puede estar presente en múltiples pedidos (relación muchos a muchos).

En casos como la relación entre pedidos y productos, se utiliza una tabla intermedia para gestionar la relación.

Diagrama Entidad-Relación (ER)

Este tipo de diagrama permite representar gráficamente las entidades y sus interacciones. Por ejemplo:

- Una entidad Clientes con atributos como ID_Cliente, Nombre y Correo Electrónico.
- Una entidad Pedidos con atributos como ID_Pedido y Fecha.
- Una relación entre Clientes y Pedidos, que muestra que un cliente puede realizar múltiples pedidos.

El diagrama conceptual sirve como una guía inicial para visualizar y organizar la base de datos antes de implementarla.

Normalización y desnormalización

Uno de los aspectos más importantes en el diseño de bases de datos es la normalización. Este proceso busca eliminar redundancias y prevenir inconsistencias en los datos. Consiste en dividir las tablas en otras más pequeñas y relacionadas de forma lógica. Sin embargo, en ciertos casos, la desnormalización puede ser útil para optimizar el rendimiento, especialmente al trabajar con grandes volúmenes de datos.

Etapas de la normalización:

1. Primera Forma Normal (1NF):

Se asegura que cada columna tenga valores únicos y atómicos, evitando listas o conjuntos de valores en una sola celda.

2. Segunda Forma Normal (2NF):

Se eliminan dependencias parciales, garantizando que todos los atributos dependan completamente de la clave primaria.

3. Tercera Forma Normal (3NF):

Se eliminan dependencias transitivas, asegurando que los atributos solo dependan de la clave primaria y no de otros atributos secundarios.

Tipos de modelos de bases de datos

1. Modelo jerárquico (años 50):

- Organiza los datos en una estructura tipo árbol.
- Existe una relación de tipo padre-hijo entre las tablas.
- Cada tabla hija está asociada exclusivamente a un único padre, aunque un padre puede tener varios hijos.

2. Modelo en red (años 60):

- Los datos se estructuran en forma de grafo, con nodos y conexiones.
- Permite que una tabla hija pueda estar vinculada a múltiples padres.

3. Modelo relacional (años 70):

- Los datos se almacenan en tablas compuestas por filas (registros) y columnas (campos).
- Supera las limitaciones del modelo jerárquico.
- Permite acceder directamente a las tablas sin necesidad de recorrer las jerarquías de padres.

4. Modelo orientado a objetos (años 90):

- Representa los datos como objetos, siguiendo un enfoque similar al de la programación orientada a objetos.
- Utiliza una estructura tridimensional.
- Es eficiente para localizar objetos individuales, pero su desempeño puede ser menor cuando se requiere trabajar con varios elementos al mismo tiempo.

Conceptos fundamentales

- Registro: Representa una fila en la tabla, donde se almacena información sobre una entidad o dato específico.
- Campo: Corresponde a una columna que guarda un tipo de dato específico para todos los registros de la tabla.
- Tabla: Estructura bidimensional formada por registros (filas) y campos (columnas). Aunque las columnas son fijas, el número de registros puede crecer sin límites.
- Base de datos relacional: Conjunto de tablas conectadas entre sí que organizan los datos de forma estructurada.
- Identificador único (clave primaria): Un valor único que identifica cada registro dentro de una tabla.
- Clave foránea: Campo que conecta dos tablas, indicando la relación entre sus registros.
- Consulta: Una instrucción escrita en SQL que permite extraer, modificar, añadir o eliminar datos en una base de datos.

Reglas de nomenclatura en SQL

Para tablas:

- Usar nombres en minúsculas.
- Evitar espacios, utilizando guiones bajos (_) en su lugar.
- Nombrar las tablas en plural o como conjuntos colectivos.
- Asegurarse de que el nombre de una tabla no coincida con el de sus campos.

Para campos:

- Utilizar nombres en singular.
- Evitar usar "id" como único identificador; es mejor un nombre más descriptivo.
- No repetir el nombre de la tabla como nombre de un campo.

- Preferir minúsculas, salvo en casos donde sea lógico emplear mayúsculas (por ejemplo, nombres propios).

Tipos de datos en SQL

SQL permite definir distintos tipos de datos para las columnas de una tabla, dependiendo de la información que se desea almacenar.

Datos numéricos:

- **INT:** Almacena números enteros. Variantes como TINYINT, SMALLINT, MEDIUMINT y BIGINT ofrecen diferentes rangos.
- **DECIMAL(p, s):** Maneja números con precisión fija, definiendo el número total de dígitos (p) y los decimales (s).
- **FLOAT y DOUBLE:** Almacenan números de punto flotante, siendo FLOAT más ligero pero menos preciso que DOUBLE.

Datos de texto:

- **CHAR(n):** Guarda cadenas de longitud fija; si son más cortas que n, se completan con espacios.
- **VARCHAR(n):** Almacena cadenas de longitud variable, ocupando solo el espacio necesario más un pequeño extra.
- **TEXT:** Permite almacenar grandes bloques de texto. Existen variantes como TINYTEXT, TEXT, MEDIUMTEXT y LONGTEXT.

Datos de fecha y hora:

- **DATE:** Formato de fecha: YYYY-MM-DD.
- **TIME:** Formato de hora: HH:MM:SS.
- **DATETIME:** Combina fecha y hora en el formato YYYY-MM-DD HH:MM:SS.

- **TIMESTAMP:** Similar a DATETIME, pero generalmente en formato UTC para registrar cambios en los datos.
- **YEAR:** Almacena años, usualmente con dos o cuatro dígitos.

Comandos básicos en SQL

1. **Crear una tabla:**

Permite definir una nueva tabla, especificando su nombre y las columnas junto con sus tipos de datos.

```
CREATE TABLE nombre_tabla (  
  
    campo_1 tipo_dato,  
  
    campo_2 tipo_dato,  
  
    ...  
  
    campo_n tipo_dato  
  
);
```

Nota: Si el nombre incluye espacios (lo cual no es recomendable), debe colocarse entre comillas dobles ("").

2. **Insertar datos:**

Agrega registros a una tabla.

```
INSERT INTO nombre_tabla (campo_1, campo_2, ..., campo_n)  
  
VALUES (valor_1, valor_2, ..., valor_n),  
  
    (valor_1, valor_2, ..., valor_n);
```

Es posible insertar múltiples registros separándolos por comas.

3. Seleccionar datos:

Recupera información de una tabla.

```
SELECT campo_1, campo_2, ..., campo_n
```

```
FROM nombre_tabla;
```

Para obtener todos los campos, se utiliza el asterisco (*):

```
SELECT *
```

```
FROM nombre_tabla;
```

Bases de datos relacionales

Las bases de datos relacionales organizan la información en tablas bidimensionales, similares a hojas de cálculo, donde cada columna representa un atributo y cada fila es un registro. Las tablas están interconectadas mediante relaciones definidas, lo que permite estructurar grandes volúmenes de datos de manera eficiente y lógica.